CMSC 420: Binary Search Trees

Justin Wyss-Gallifent

January 16, 2026

1 Problems **2**

1 Problems

1. Consider the following tree:

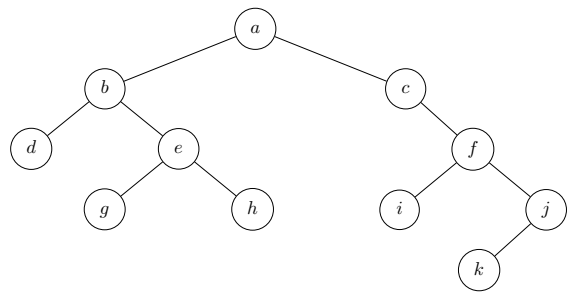


Figure 1: A Binary Search Tree

(a) List the nodes according to a pre-order traversal.
Solution:
A pre-order traversal yields: a, b, d, e, g, h, c, f, i, j, k

(b) List the nodes according to an in-order traversal.
Solution:
An in-order traversal yields: d, b, g, e, h, a, c, i, f, k, j

(c) List the nodes according to a post-order traversal.
Solution:
A post-order traversal yields: d, g, h, e, b, i, k, j, f, c, a

(d) Redraw the tree and include in-order threads.
Solution:
The result is:

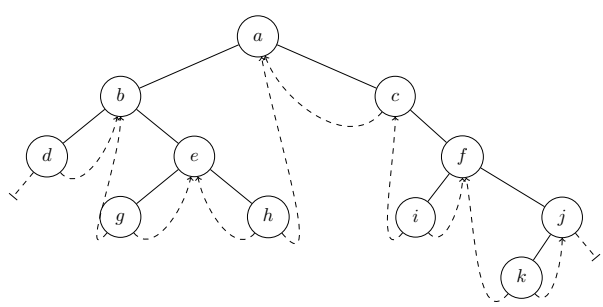


Figure 2: In-Order Threads

2. Define b_h to be the number of distinct binary tree structures with height h. We don't care about the contents, just the overall structure.

   (a) What is b_0 and why?

   Solution:
   A tree with height $h = 0$ is a single node, and there is only one such tree, so $b_0 = 1$.

   (b) What is b_1 and why?

   Solution:
   A tree with height $h = 1$ is a single node with either a left child and no right child, a right child but no left child, or two children. Thus $b_1 = 3$.

   (c) Prove that for $h \geq 2$ we have:

$$b_h = b_{h-1}^2 + 2b_{h-1}\left(1 + b_0 + b_1 + ... + b_{h-2}\right)$$

   Solution:
   A binary tree with height h has exactly one of the following forms:

   - A node with two child subtrees, each having height $h-1$. There are b_{h-1}^2 such structures.
   - A node with a left child subtree having height $h-1$ and no right child subtree. There are b_{h-1} such structures.
   - A node with a right child subtree having height $h-1$ and no left child subtree. There are b_{h-1} such structures.
   - A node with a left child subtree having height $h-1$ and a right child subtree having height between 0 and $h-2$ inclusive. There are $b_{h-1}(b_0 + b_1 + ...b_{h-2})$ such structures.
   - A node with a right child subtree having height $h-1$ and a left child subtree having height between 0 and $h-2$ inclusive. There are $b_{h-1}(b_0 + b_1 + ... + b_{h-2})$ such structures.

   The total is then the sum of these, which is what is given.

   Alternate Solution:
   An empty tree has height $h = -1$ then there is only one such structure and so $h_{-1} = 1$. A binary tree with height h must have either left child subtree or right child subtree (or both) with height $h-1$. If the left child subtree has height $h-1$ then the right child subtree may have height from -1 to $h-1$ inclusive (-1 would be no right child subtree) and there are $b_{h-1}(b_{-1} + b_0 + ... + b_{h-1})$ such structures. Symmetrically we get the same answer if the right child subtree has height $h-1$ and the left child subtree may have height from -1 to $h-1$. The final result is not the sum of these, however, since we've double-counted the case when both child subtrees have height $h-1$ so we must subtract b_{h-1}^2. This yields the same result.

(d) Use this to calculate b_h for $n = 2, 3, 4, 5$.

Solution:

We have:

$$b_2 = b_1^2 + 2b_1(1 + b_0) = 3^2 + 2(3)(1 + 1) = 21$$
$$b_3 = b_2^2 + 2b_2(1 + b_0 + b_1) = 21^2 + 2(21)(1 + 1 + 3) = 651$$
$$b_4 = b_3^2 + 2b_3(1 + b_0 + b_1 + b_2) = 651^2 + 2(651)(1 + 1 + 3 + 21) = 457653$$
$$b_5 = b_4^2 + 2b_4(1 + b_0 + b_1 + b_2 + b_3)$$

3. A trinary tree is a tree in which each node has between 0 and 3 children (call these the left child, middle child, and right child). Define $T(h)$ for $h \geq -1$ to be the number of distinct trinary tree structures with height h. We don't care about the contents, just the overall structure.

   (a) What is $T(-1)$ and why?

   Solution:
   A tree with height $h = -1$ is empty, and there is only one such tree, so $T(-1) = 1$.

   (b) What is $T(0)$ and why?

   Solution:
   A tree with height $h = 0$ is a single node, and there is only one such tree, so $T(0) = 1$.

   (c) What is $T(1)$ and why?

   Solution:
   A tree with height $h = 1$ is a single node with either only a left child, only a middle child, only a right child, only left and middle children, only left and right children, only middle and right children, or all three children.
   Thus $T(1) = 7$.

   (d) Prove that for $h \geq 1$ we have:

   $$T(h) = 3T(h-1)(T(-1)+T(0)+...+T(h-2))^2+3T(h-1)^2(T(-1)+T(0)+...+T(h-2))+T(h-1)^3$$

   Solution:
   A trinary tree with height h consists of a root node with the condition that at least one child subtree has height $h-1$.

   - If only the left child subtree has height $h-1$ then there are $T(h-1)(T(-1)+T(0)+...+T(h-2))^2$ such structures.
   - If only the middle child subtree has height $h-1$ then there are $T(h-1)(T(-1)+T(0)+...+T(h-2))^2$ such structures.
   - If only the right child subtree has height $h-1$ then there are $T(h-1)(T(-1)+T(0)+...+T(h-2))^2$ such structures.
   - If only the left and middle child subtrees have height $h-1$ then there are $T(h-1)^2(T(-1)+T(0)+...+T(h-2))$ such structures.
   - If only the left and right child subtrees have height $h-1$ then there are $T(h-1)^2(T(-1)+T(0)+...+T(h-2))$ such structures.
   - If only the middle and right child subtrees have height $h-1$ then there are $T(h-1)^2(T(-1)+T(0)+...+T(h-2))$ such structures.
   - If all three child subtrees have height $h-1$ then there are $T(h-1)^3$ such structures.

   The answer is then the sum of these:

   $$3T(h-1)(T(-1)+T(0)+...+T(h-2))^2+3T(h-1)^2(T(-1)+T(0)+...+T(h-2))+T(h-1)^3$$

(e) Use this to calculate $T(2)$ for $n = 2$.

Solution:

We have:

$$\begin{aligned} T(2) &= 3T(1)(T(-1) + T(0))^2 + 3T(1)^2(T(-1) + T(0)) + T(1)^3 \\ &= 3(7)(1+1)^2 + 3(7)^2(1+1) + (7)^3 \\ &= 721 \end{aligned}$$

4. For a binary tree T with n nodes define $d(x, T)$ to be the depth of the node x (the root is depth 0) and define:

$$P(T) = \sum_{x \in T} d(x, T)$$

If T_L and T_R are the two subtrees of the root of a tree T, prove that:

$$P(T) = P(T_L) + P(T_R) + n - 1$$

Solution:
Suppose n_L and n_R are the number of nodes in those trees, then:

$$\begin{aligned}
P(T) &= \sum_{x \in T} d(x, T) \\
&= d(root, T) + \sum_{x \in T_L} d(x, T) + \sum_{x \in T_R} d(x, T) \\
&= 0 + n_L + \sum_{x \in T_L} d(x, T_L) + n_r + \sum_{x \in T_R} d(x, T_R) \\
&= \sum_{x \in T_L} d(x, T_L) + \sum_{x \in T_R} d(x, T_R) + n_L + n_R \\
&= \sum_{x \in T_L} d(x, T_L) + \sum_{x \in T_R} d(x, T_R) + n - 1 \\
&= P(T_L) + P(T_R) + n - 1
\end{aligned}$$

5. Arrange the numbers 1 through 10 such that when inserted into a binary search tree (which is originally empty) the tree is a complete binary tree after each and every insertion. Briefly explain the logic you used to come up with your answer.

   Solution:
   The only solution is in fact:

$$7, 4, 9, 2, 6, 8, 10, 1, 3, 5$$

   Logic may vary.

6. (a) Consider the binary tree structure:

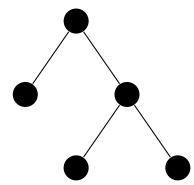


Figure 3: Binary Tree Structure

Assign the list $A = [1, 2, 3, 4, 5]$ as node keys so the result is a binary search tree.

Solution:

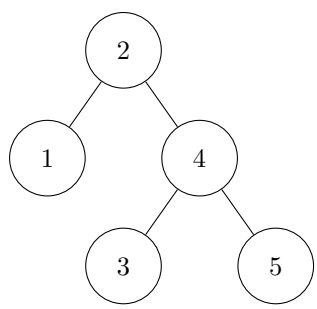


Figure 4: Assigned Binary Tree Structure

(b) Given an arbitrary binary tree structure with n nodes explain algorithmically (how would you do it?) why it is always possible to select n distinct integers and assign them as node keys within the structure so that the result is a binary search tree.

Solution:

Let $A = [1, 2, ..., n]$. Let N_L be the number of nodes in the left subtree of the root and let N_R be the number of nodes in the right subtree of the root. The smallest N_L elements of A must be assigned in the left subtree while the largest N_R elements of A must be assigned in the right subtree. The leftover element must be assigned to the root. Follow this process recursively with the left and right subtrees.

7. Here is the pseudocode for an inorder traverse:

```
function inorder(x):
    if x != NULL:
        inorder(x.left)
        print(x.key)
        inorder(x.right)
    end if
end function
```

Prove by strong induction on $n \geq 0$, the number of nodes in a tree, that if `inorder(x)` is called where `x` is the root that the output is the keys in increasing order. Assume the keys are all distinct.

Solution:
Base Case: For $n = 0$ we print nothing which is the same as the keys in increasing order.
Inductive Step: Assume the statement is true for all trees with $1 \leq k \leq n$ nodes. Suppose we have a tree with $n + 1$ nodes for such that x is the root. We make a call to the left child which by the inductive hypothesis prints the left child's keys in increasing order. By the structure of a binary search tree these are all less than x's key. Then we print x's key. We then make a call to the right child which again by the inductive hypothesis prints the right child's keys in increasing order. Again by the structure of a binary search tree these are all greater than x's key. The result is then that we have printed all the keys in increasing order.

8. Define f_h to be the number of distinct full binary tree structures with height h. We don't care about the contents, just the overall structure.

   (a) Draw all possible distinct full binary trees with height $h = 0$ and conclude what the value of f_0 is.

   Solution:
   We have only the following, so $f_0 = 1$:

Figure 5: Full Binary Tree Structures with Height 0

   (b) Draw all possible distinct full binary trees with height $h = 1$ and conclude what the value of f_1 is.

   Solution:
   We have only the following, so $f_1 = 1$:

Figure 6: Full Binary Tree Structures with Height 1

   (c) Draw all possible distinct full binary trees with height $h = 2$ and conclude what the value of f_2 is.

   Solution:
   We have only the following, so $f_1 = 3$:

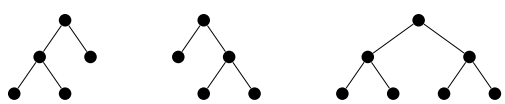


Figure 7: Full Binary Tree Structures with Height 2

   (d) Prove that for $h \geq 2$ we have:

$$f_h = f_{h-1}^2 + 2f_{h-1}\left(f_0 + f_1 + ... + f_{h-2}\right)$$

   Solution:
   A full binary tree with height $h \geq 1$ has exactly one of the following forms:

   - A node with two child full subtrees, each having height $h - 1$. There are f_{h-1}^2 such structures.
   - A node with a left child full subtree having height $h - 1$ and a right child subtree having height between 0 and $h - 1$ inclusive. There are $f_{h-1}(f_0 + f_1 + ...f_{h-2})$ such structures.

- A node with a right child full subtree having height $h-1$ and a left child subtree having height between 0 and $h-1$ inclusive. There are $f_{h-1}(f_0 + f_1 + ... + f_{h-2})$ such structures.

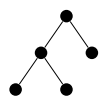
The total is then the sum of these, which is what is given.

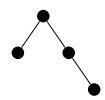
9. For a binary tree T containing n nodes there are ways to measure how balanced it is with a single number. One of them is the fill ratio:

$$FR(T) = \frac{n}{2^{H(T)+1} - 1}$$

The fill ratio essentially asks - what is the ratio of the current number of nodes to the number of nodes which would be in the tree if it had the same height and was perfect (full on every level).

(a) Calculate this value for the following trees:

i. 

ii. 

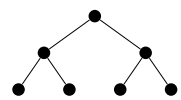
iii. 

Figure 8: Fill Ratio Trees

Solution:
We have:

i. $h(T) = 2$ and so $FR(T) = \frac{5}{2^{2+1}-1} = \frac{5}{7} \approx 0.7143$
ii. $h(T) = 2$ and so $FR(T) = \frac{4}{2^{2+1}-1} = \frac{4}{7} \approx 0.5714$
iii. $h(T) = 2$ and so $FR(T) = \frac{5}{2^{2+1}-1} = \frac{7}{7} = 1$

(b) For a binary tree with n nodes what are the maximum and minimum fill ratios? Calculate these for each of the values $n = 1, ..., 10$, approximating to four digits after the decimal point for each.

Solution:
The maximum fill ratio occurs when the tree has minimum height (keeping the denominator small) so $H(T) = \lfloor \lg n \rfloor$ and so:

$$FR(T) = \frac{n}{2^{H(T)+1} - 1} = \frac{n}{2^{\lfloor \lg n \rfloor + 1} - 1}$$

The minimum fill ratio occurs when the tree has maximum height (keeping the denominator large) which occurs when the tree is a linked list so $H(T) = n - 1$ and so:

$$FR(T) = \frac{n}{2^{H(T)+1} - 1} = \frac{n}{2^{n-1+1} - 1} = \frac{n}{2^n - 1}$$

Here are the values:

n	Minimum	Maximum
1	1	1
2	0.6667	0.6667
3	0.4286	1
4	0.2667	0.5714
5	0.1613	0.7143
6	0.0952	0.8571
7	0.0551	1
8	0.0314	0.5333
9	0.0176	0.6
10	0.0098	0.6667

(c) For a binary tree with height h what are the maximum and minimum fill ratios? Calculate these for each of the height $h = 1, ..., 10$, approximating to four digits after the decimal point for each.

Solution:

The maximum fill ratio occurs when the tree has the maximum number of nodes (keeping the numerator large) which occurs when the tree is perfect so $n = 2^{h+1} - 1$ and so:

$$FR(T) = \frac{n}{2^{H(T)+1} - 1} = \frac{2^{h+1} - 1}{2^{h+1} - 1} = 1$$

The minimum fill ratio occurs when the tree has the minimum number of nodes (keeping the numerator small) which occurs when the tree is a linked list so $n = h + 1$ and so:

$$FR(T) = \frac{n}{2^{H(T)+1} - 1} = \frac{h + 1}{2^{h+1} - 1}$$

Here are the values:

h	Minimum	Maximum
1	0.6667	1
2	0.4286	1
3	0.2667	1
4	0.1613	1
5	0.0952	1
6	0.0551	1
7	0.0314	1
8	0.0176	1
9	0.0098	1
10	0.0054	1

10. Suppose a binary tree contains n nodes.

(a) Write down an expression for the minimum possible height of the tree, with justification.

Solution: A tree with height h has at most $2^{h+1} - 1$ nodes, when it is perfect.
Thus $n \leq 2^{h+1} - 1$ and so $h \geq \lg(n+1) - 1$. Thus the minimum possible height would be $\lceil \lg(n+1) - 1 \rceil$.

(b) Write down an expression for the maximum possible height of the tree, with justification.

Solution:
The maximum possible height would be when the tree is a linked list. With n nodes this would be height $n - 1$.

11. Draw a single binary tree which has the following two properties:

- For every node x we have $\text{height}(x.left) - \text{height}(x.right) \in \{-1, 0, 1\}$.
- There are two leaves whose levels differ by 2.

Solution:
Here is an example:

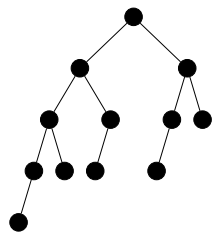


Figure 9: Example Binary Tree

12. A binary search tree has the following structure:

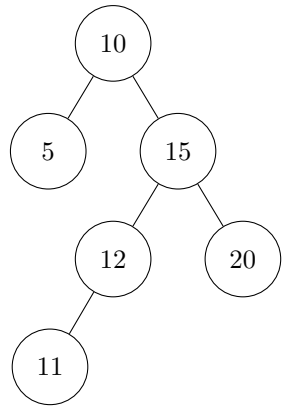


Figure 10: A Binary Search Tree

Suppose we delete a key and the height of the tree changes. Various choices of key and/or use of IOP/IOS replacements could make this happen. What are the possibilities? Explain each.

Solution:
Here are the options:

- We could delete 11.
- We could delete 12.
- We could delete 15 and use the IOP which is 12 as the replacement.
- We could delete 10 and use the IOS which is 11 as the replacement.

13. Suppose we have a binary search tree containing n nodes. We pick a key at random with all keys equally likely and delete it. Suppose that deleting a node on level k takes time exactly k (accounting for all necessary work).

    What structure could the tree have which would guarantee that the average-case time to do this would be $\Theta(n)$? Justify.

    Solution:
    If the tree is a linked list this will happen. This is because the average time will be:

$$\sum_{k=0}^{n-1} \frac{1}{n} k = \frac{1}{n} \cdot \frac{(n-1)(n)}{2} = \Theta(n)$$

14. Suppose we have a perfect binary search tree with L levels indexed $0, ..., L-1$. Thus it has $n = 2^L - 1$ nodes. Suppose that deleting a node on level k takes time exactly k (accounting for all necessary work).

(a) For an unknown $0 \leq k \leq L-1$ what is the probability that we pick a key on level k?

Solution:

There are 2^k keys on level k so the probability is:

$$\frac{2^k}{n}$$

(b) We pick a key at random with all keys equally likely and delete it. How long do we expect it to take to delete the key?

Hint: Sum over the levels, and you may need to find a formula for this sum.

Solution:

We expect it to take time:

$$\begin{aligned}\sum_{k=0}^{L-1} \frac{2^k}{n}(k) &= \frac{1}{n}\sum_{k=0}^{L-1} k2^k \\ &= \frac{1}{n}\left((L-2)2^L + 2\right)\end{aligned}$$

Since $n = 2^L - 1$ we know that $L = \lg(n+1)$ and $2^L = n+1$ and so the result is:

$$\frac{1}{n}\left((\lg(n+1) - 2)(n+1) + 2\right)$$

15. Suppose a binary tree has the property that each node's key is larger than the key in its left child (if one exists) and smaller than the key in its right child (if one exist). Is this same as being a BST? Explain.

Solution:
No, for example the following tree satisfies the above condition but is not a BST:

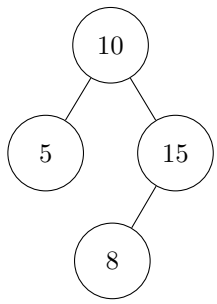


Figure 11: A Binary Search Tree

16. Consider this binary search tree:

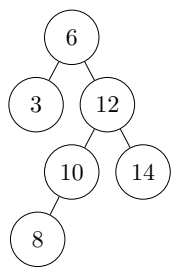


Figure 12: A Binary Search Tree

The following are separate questions about this tree.

(a) We run `search(x)` where x is a key in the tree chosen randomly with all keys equally likely. If it takes time $k+1$ to find a key on level k how much time do we expect the search to take? Simplify.

Expected Time	5/2

(b) We run `insert(x)` where x is an integer, $1 \leq x \leq 15$, and x differs from all keys in the tree. The height of the tree does not change. What could x be? List all values in increasing order.

Possible x	$1, 2, 4, 5, 11, 13, 15$

(c) Suppose we delete a key and the height of the tree does not change. Various choices of key and/or use of IOP/IOS replacements could make this happen. What are the possibilities? Explain each. You don't need pictures, just list the keys which could be deleted and, if a replacement is used, what that would be and whether it is the IOP or IOS.

Solution:
Here are the options:

- We could delete 3.
- We could delete 14.
- We could delete 12 and use the IOS which is 14 as the replacement.
- We could delete 6 and use the IOP which is 3 as the replacement.

17. Consider all binary trees which have the property that for every node the height of the left and right subtrees differ by at most 2. Denote by N_h the number of different trees with this property. We don't care about the contents, just the overall structure.

(a) Show that $N_2 \geq 3$ by drawing exactly three such trees.

Solution:

Answers vary.

(b) Consider the following possibilities:

(A) $N_h = N_{h-1}^2 + N_{h-1}N_{h-2} + N_{h-1}N_{h-3}$

(B) $N_h = N_{h-1}^2 + 2N_{h-1}N_{h-2} + 2N_{h-1}N_{h-3}$

(C) $N_h = 2N_{h-1}N_{h-1} + 2N_{h-1}N_{h-2} + 2N_{h-1}N_{h-3}$

(D) None of the above.

Which of these is true?

Which? A or B or C or D?	B